

Conversational Q&A Chatbot

The Big Picture: A Smart Chatbot with Memory

The main goal of this code is to create a **Q&A Chatbot** that can:

1. **Answer questions** based on a specific set of documents (like a blog post).
2. **Remember past conversations** so you can ask follow-up questions without repeating yourself (this is the "memory" part).

◆ What This Code Builds?

A chatbot that:

- Scrapes a webpage.
- Splits the text into chunks.
- Embeds those chunks using a vector model.
- Stores them in a vector database (Chroma).
- Retrieves relevant chunks during a chat.
- Remembers previous questions and answers.

1. Load Keys and LLM

```
import os
from dotenv import load_dotenv
load_dotenv()
from langchain_groq import ChatGroq

groq_api_key=os.getenv("GROQ_API_KEY")

llm=ChatGroq(groq_api_key=groq_api_key,model_name="Llama3-8b-8192")
```

llm

```
ChatGroq(client=<groq.resources.chat.completions.Completions object at 0x000002234054A8C0>, asyn
```

2. 🧠 Load Embeddings Model

```
os.environ['HF_TOKEN'] = os.getenv("HF_TOKEN")
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
```

Import Libraries:

```
from langchain_chroma import Chroma
from langchain_community.document_loaders import WebBaseLoader
from langchain_core.prompts import ChatPromptTemplate
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_c
hain
```

3. 🕸 Load Web Page

Web page → <https://lilianweng.github.io/posts/2023-06-23-agent>

```
from langchain_community.document_loaders import WebBaseLoader
import bs4

loader = WebBaseLoader(
    web_paths=("https://lilianweng.github.io/posts/2023-06-23-agent/"),
    bs_kwargs=dict(parse_only=bs4.SoupStrainer(class_=("post-content", "pos
```

```
t-title", "post-header"))
)
docs = loader.load()
```

Output:

```
[Document(metadata={'source': 'https://lilianweng.github.io/posts/2023-06-23-agent/'}, page_content='\n\n    LLM Powered Autonomous Agents\n    \nDate: June 23, 2023 | Estimated Reading Time: 31 min | Author: Lilian Weng
...]
```

- `WebBaseLoader` is a tool from LangChain that **fetches content from a web URL**.
- `web_paths=...` is the URL of the blog post you want to load
- `bs_kwargs=dict(parse_only=bs4.SoupStrainer(...))` : This is a clever trick using `BeautifulSoup` . It tells the loader: "Don't grab *everything* on the page (like navigation menus or ads). Only get the main article content, the title, and the header." This ensures we only get useful information.
 - `post-title` → usually the title of the blog
 - `post-header` → usually subtitle or metadata
 - `post-content` → the main article body

4. ✂ Split into Chunks

```
text_splitter=RecursiveCharacterTextSplitter(chunk_size=1000,chunk_overlap=200)
splits=text_splitter.split_documents(docs)
```

Splits text: Helps ensure each chunk is manageable (e.g. fits token limits).

5. 🧠 Create Vector Store

```
vectorstore=Chroma.from_documents(documents=splits,embedding=embeddings)
```

```
retriever=vectorstore.as_retriever()  
retriever
```

```
VectorStoreRetriever(tags=['Chroma', 'HuggingFaceEmbeddings'], vectorstore=<langchain_chroma.vectorstores.Chroma ol
```

- **Chroma:** In-memory vector database to store embeddings.
- **as_retriever():** Wraps it in a standard interface for retrieval.

6. Create Prompt Template

```
## Prompt Template
```

```
system_prompt = (  
    "You are an assistant for question-answering tasks. "  
    "Use the following pieces of retrieved context to answer "  
    "the question. If you don't know the answer, say that you "  
    "don't know. Use three sentences maximum and keep the "  
    "answer concise."  
    "\n\n"  
    "{context}"  
)
```

```
prompt = ChatPromptTemplate.from_messages(  
    [  
        ("system", system_prompt),  
        ("human", "{input}")  
    ]  
)
```

- **system_prompt**: This is the "rulebook" we give to our AI
- **{context}**: Filled in from documents during retrieval.

7. Create Retrieval Chain

```
from langchain.chains import create_stuff_documents_chain, create_retrieval_chain
```

```
question_answer_chain = create_stuff_documents_chain(llm, prompt)
rag_chain = create_retrieval_chain(retriever, question_answer_chain)
```

```
response = rag_chain.invoke({"input": "What is Self-Reflection"})
```

```
{'input': 'What is Self-Reflection',
 'context': [Document(id='138a3eb6-15d7-4efd-8381-2762cfb80d52', metadata={'source': 'https://lilianweng.github.io/posts/2023-06-23-agent'}, score=0.95), Document(id='0106b350-fd7a-4732-b231-218ce62737b5', metadata={'source': 'https://lilianweng.github.io/posts/2023-06-23-agent'}, score=0.9), Document(id='77af004b-4f7a-4e77-95a3-bc26de7245a9', metadata={'source': 'https://lilianweng.github.io/posts/2023-06-23-agent'}, score=0.85), Document(id='f90b568d-3f2f-4b35-bf63-e3e5abd4d9f', metadata={'source': 'https://lilianweng.github.io/posts/2023-06-23-agent'}, score=0.8)],
 'answer': 'Self-Reflection is a vital aspect of autonomous agents that allows them to improve iteratively by refining past actions and decisions based on feedback and self-analysis.'}
```

You could also write:

```
rag_chain1 = {
    "context": retriever,
    "input": RunnablePassthrough()
} | prompt | llm # llm = ChatGroq/Llama3
```

```
response1= rag_chain1.invoke("What is Self-Reflection")
```

```
response1.content
```

```
'Self-reflection is a vital aspect that allows autonomous agents to improve iteratively by refining past actions and decisions based on feedback and self-analysis.'
```

- `question_answer_chain = create_stuff_documents_chain (llm, prompt)` : This is a **mini-chain**.
 - It takes the `llm` (AI brain) and the `prompt` (rulebook).

- Its job is to take the retrieved documents and the user's question, "stuff" them into the `{context}` and `{input}` placeholders of the `prompt`, and then send that complete message to the `llm` for an answer.
- Behind the scenes, this chain:
 1. Uses the `retriever` to get some documents (like paragraphs).
 2. Takes the **text from those documents**.
 3. Fills it into `{context}` in the `system_prompt`.
- `rag_chain = create_retrieval_chain (retriever, question_answer_chain)` : This is the **complete RAG chain**.
 - It says: "First, use the `retriever` to find relevant documents. Then, pass those documents (along with the original question) to the `question_answer_chain` to get the final answer."

```
response['answer']
```

Output:

'Self-Reflection is a vital aspect of autonomous agents that allows them to improve iteratively by refining past action decisions and correcting previous mistakes. It enables agents to learn from their experiences and adjust their behavior accordingly.'

8. Add Chat History

```
from langchain.chains import create_history_aware_retriever
from langchain_core.prompts import MessagesPlaceholder
```

`create_history_aware_retriever` : This is a special LangChain tool that helps the retriever understand the conversation history.

MessagesPlaceholder : This is a specific type of placeholder in a prompt that's designed to hold a list of past chat messages.

```
from langchain_core.prompts import MessagesPlaceholder

contextualize_q_system_prompt = (
    "Given a chat history and the latest user question "
    "which might reference context in the chat history, "
    "formulate a standalone question which can be understood "
    "without the chat history. Do NOT answer the question, "
    "just reformulate it if needed and otherwise return it as is."
)
contextualize_q_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", contextualize_q_system_prompt),
        MessagesPlaceholder("chat_history"),
        ("human", "{input}"),
    ]
)
```

- You can write anything instead of `chat_history` , but `chat_history` is recommended.

```
from langchain.chains import create_history_aware_retriever

history_aware_retriever=create_history_aware_retriever(llm,retriever,contextua
lize_q_prompt)

history_aware_retriever
```

```
RunnableBinding(bound=RunnableBranch(branches=[(RunnableLambda(lambda x: not x.get('chat_history', False)), Runn
| VectorStoreRetriever(tags=['Chroma', 'HuggingFaceEmbeddings'], vectorstore=<langchain_chroma.vectorstores.Chro
| ChatGroq(client=<groq.resources.chat.completions.Completions object at 0x000001B35388A890>, async_client=<groq
| StrOutputParser())
| VectorStoreRetriever(tags=['Chroma', 'HuggingFaceEmbeddings'], vectorstore=<langchain_chroma.vectorstores.Chro
```



We're upgrading the `retriever` → `history_aware_retriever`

Remember:

- `retriever = vectorstore.as_retriever()`
 - This is the normal retriever that fetches documents based on questions.
- `contextualize_q_prompt` :
 - This is a **special prompt** that teaches the LLM how to rewrite the follow-up question.



Why do we need this?

Let's say a conversation goes like this:



: What is LangChain?



: How is it different from LlamaIndex?

The **second question** ("*How is it different from LlamaIndex?*") makes sense only if we remember the previous question was about **LangChain**.

- A **normal retriever** doesn't remember that.
- But a **history-aware retriever uses past messages** to understand follow-up questions.

9. 🧠 Final RAG Chain With History

- Let's rewrite the prompt with different name

```
system_prompt = (
    "You are an assistant for question-answering tasks. "
```



```

"Use the following pieces of retrieved context to answer "
"the question. If you don't know the answer, say that you "
"don't know. Use three sentences maximum and keep the "
"answer concise."
"\n\n"
"{context}"
)

qa_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system_prompt),
        MessagesPlaceholder("chat_history"),
        ("human", "{input}"),
    ]
)

```

```

question_answer_chain=create_stuff_documents_chain(llm,qa_prompt)
rag_chain=create_retrieval_chain(history_aware_retriever,question_answer_chain)

```



We replaced `retriever` with `history_aware_retriever`

10. **Manually** Update Chat History

```

question= "What is Self-Reflection?"

response1= rag_chain.invoke({"input": question, "chat_history": chat_history})

print(response1['answer'])

```

Output:

Self-Reflection is a process that allows autonomous agents to improve iteratively by refining past action decisions and correcting previous mistakes. It is a vital aspect that enables agents to learn from their experiences and improve their performance over time.

Add History:

```
from langchain_core.messages import AIMessage, HumanMessage
```

```
chat_history = []
```

```
chat_history.extend([
    HumanMessage(question),
    AIMessage(response1['answer'])
])
```

```
question2 = "Tell me more about it"
```

```
response2 = rag_chain.invoke({"input": question2, "chat_history": chat_history})
```

```
print("=====")
```

```
print(response2['answer'])
```

```
=====
I don't have more information on Self-Reflection beyond what is already provided in the context.
```

Output:

Self-Reflection is a process that allows autonomous agents to improve iteratively by refining past action decisions and correcting previous mistakes. It plays a crucial role in real-world tasks where trial and error are inevitable.

=====

I don't know. The provided context only mentions Self-Reflection as a process that allows autonomous agents to improve iteratively by refining past action decisions and correcting previous mistakes, but it does not provide further information.

11. 🧠 Add Automatic Chat History

```
from langchain.chains import create_stuff_documents_chain, create_retrieval_chain
```

```
question_answer_chain = create_stuff_documents_chain(llm, prompt)
```

```
rag_chain = create_retrieval_chain(retriever, question_answer_chain)
```

```
from langchain_community.chat_message_histories import ChatMessageHistory
```

```
from langchain_core.runnables.history import RunnableWithMessageHistory
```

```
store = {}
```

```
def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]
```

```

conversational_rag_chain = RunnableWithMessageHistory(
    rag_chain,
    get_session_history,
    history_messages_key = "chat_history",
    input_messages_key= "input",
    output_messages_key="answer"

)

```

```

conversational_rag_chain.invoke(
    {"input": "What is Task Decomposition?"},
    config={
        "configurable": {"session_id": "abc123"}
    }, # constructs a key "abc123" in `store`.
)["answer"]

```

```
'Task decomposition is the process of breaking down a complex task into sm
```

```

conversational_rag_chain.invoke(
    {"input": "What are common ways of doing it?"},
    config={"configurable": {"session_id": "abc123"}},
)["answer"]

```

```
'Common ways of doing task decomposition include using simple prompting, task-specific instructions, and relying o
```

store

Output:

```
{'abc123': InMemoryChatMessageHistory(messages=[HumanMessage(content='What is Task Decomposition?', additional_kwargs={}, response_metadata={}), AIMessage(content='Task decomposition is the process of breaking down a complex task into smaller, more manageable subtasks. This can be done using techniques such as prompting the LLM with simple instructions like "Steps for XYZ" or using task-specific instructions, or by relying on an external classical planner to do long-horizon planning.', additional_kwargs={}, response_metadata={}),
```

```
HumanMessage(content='What are common ways of doing it?', additional_kwargs={}, response_metadata={}), AIMessage(content='Common ways of doing task decomposition include using simple prompting, task-specific instructions, and relying on an external classical planner.', additional_kwargs={}, response_metadata={})])}]}
```